
复合数据类型 结构体

复习

- | | |
|------------------|----|
| 1. 指针概述 | 了解 |
| 2. 指针定义与初始化 | 熟悉 |
| 3. 指针和数组 | 重点 |
| 4. 二级指针 | 熟悉 |
| 5. 指针和函数 | 重点 |
| 6. 指针与字符串 | 重点 |
| 7. NULL指针和void指针 | 熟悉 |
| 8. 指针与引用的区别 | 重点 |

目录

- | | |
|-----------------|----|
| 1. 结构体概述 | 了解 |
| 2. 结构体的定义和初始化 | 重点 |
| 3. 结构体指针 | 重点 |
| 4. 结构体数组 | 重点 |
| 5. 结构体做函数参数 | 重点 |
| 6. 共用体 | 熟悉 |
| 7. 枚举类型 | 熟悉 |
| 8. 用typedef定义类型 | 熟悉 |

结构体概述

如果想保存学生的学号，姓名，年龄，各科成绩等信息，那种数据类型可以把这些数据作为一个整体保存起来？

结构体概述

结构体(struct)是具有**相同 类型 或不同类型**的数据构成的数据集合。

C++ 中，结构体(struct)指的是一种数据结构，是 C++ 中聚合数据类型(aggregate data type)的一类。结构体可以被声明为 **变量、指针 或 数组** 等，用以实现较复杂的数据结构。

结构体同时也是一些元素的集合，这些元素称为结构体的成员(member)，且这些成员可以为不同的类型，成员一般用名字访问。

结构体的定义和初始化

C++结构体定义 语法

```
struct structName{  
    type name1;  
    type name2;  
    type name3;  
};
```

用 struct 定义了一个结构体 structName，该结构体有三个字段，字段名分别为 name1、name2 和 name3。定义结构体时，每个字段名后面都需要使用 ; 结尾;最后的大括号后面，同样需要使用 ; 结尾，**如果没有分号，则程序会报错。**

结构体的定义和初始化

```
struct student {  
    string name;  
    int age;  
    double score;  
};
```

结构体变量定义

```
struct 结构体名 结构体变量名;  
student s1,s2,s3;
```

```
struct student {  
    string name;  
    int age;  
    double score;  
}s1,s2,s3;
```

结构体的定义和初始化

```
#include <iostream>
using namespace std;
struct student {
    string name;
    int age;
    double score;
};
int main() {
```

```
    struct student s1;
    s1.name = "张三";
    s1.age = 20;
    s1.score = 99.5;
```

```
    return 0;
```

```
}
```

```
student s1 = {"张三", 20, 99.5};
```



结构体成员访问

```
#include <iostream>
using namespace std;
struct student {
    string name;
    int age;
    double score;
};
int main() {

    student s1 = {"张三", 20, 99.5};
    cout << s1.name << endl;
    cout << s1.age << endl;
    cout << s1.score << endl;
    return 0;
}
```



访问结构体成员

结构体默认值

在C++11标准后，可以直接在定义结构体时给成员赋初始值

```
struct student {  
    string name="C++";  
    int age{20};  
    double score{99.5};  
};
```

结构体中定义函数

语法

```
struct structName{  
    type name1;  
    type name2;  
    type name3;  
    type funcName(type val1 , type val2) {  
        return val;  
    }  
};
```

结构体中定义函数

```
struct student {  
    string name;  
    int age;  
    double score;  
    void printInfo(){ //打印学生信息  
        cout << name << endl;  
        cout << age << endl;  
    }  
};
```

结构体指针

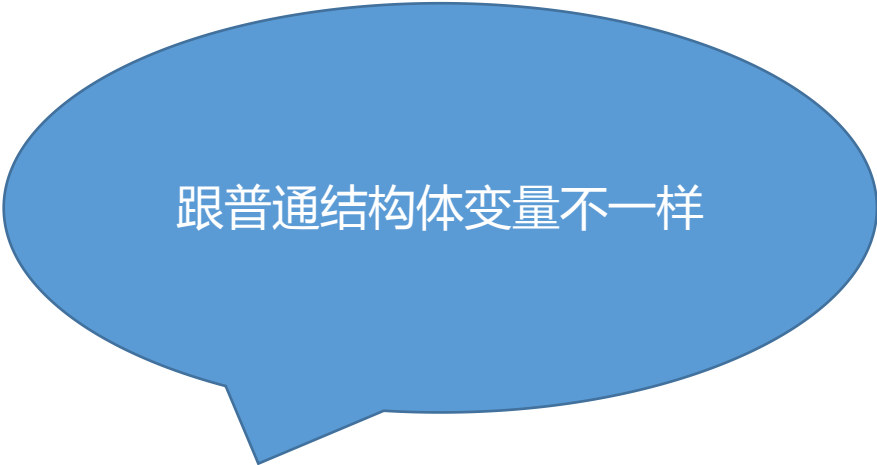
指针 可以存放任意的 数据类型 的地址，比如float、 int、 long、 double，也可以存放 结构体 的地址，存放结构体地址的指针，称为结构体指针。

语法

```
struct structName *pName;
```

定义了一个结构体指针 pName，该指针存放的是结构体 structName 的地址。

访问结构体指针成员



跟普通结构体变量不一样

语法

`pName->fileld;`

如果是结构体指针，要访问其成员，不再是使用点，而是使用 ->。

访问结构体指针成员

语法

`(*pName).fileId;`

使用 * 的方式，访问结构体成员，必须要使用 () 改变运算符的优先级，否则，会出错。

访问结构体指针成员案例

```
student s1;  
student* ps1 = &s1;  
ps1->name = "诸葛亮";  
(*ps1).age = 20;  
cout << ps1->name << endl;  
cout << (*ps1).age << endl;
```

结构体数组

数组可以存放任意的数据类型，比如 int、double，也可以存放结构体，存放结构体的数组，称为结构体数组。

语法

```
struct structName arrName[len];
```

定义了一个结构体数组 arrName，该数组中每一个元素的类型都是一个 structName 类型的结构体，数组的长度为 len。

结构体数组 案例

```
student arrstu[3];  
for (int i = 0; i < 3; i++) { //输入  
    cin >> arrstu[i].name >> arrstu[i].age >> arrstu[i].score;  
}  
for (int i = 0; i < 3; i++) { //输出  
    cout << arrstu[i].name << " " << arrstu[i].age << " " << arrstu[i].score << endl;  
}
```

结构体数组 案例

```
student arrstu[3] = {  
    {"name1",20,90},  
    {"name2",21,92},  
    {"name3",22,99},  
};  
for (int i = 0; i < 3; i++) { //输出  
    cout << arrstu[i].name << " " << arrstu[i].age << " " << arrstu[i].score << endl;  
}
```

结构体做函数参数

语法

```
type funcName(structName structVal){  
    return val;  
}
```



结构体指针
结构体数组
也可以做参数

使用结构体变量作为函数的实参时,采用的是值传递,会将结构体变量所占内存单元的内容全部顺序传递给形参,形参必须是同类型的结构体变量。

结构体做函数参数 案例

```
#include <iostream>
using namespace std;
struct student {
    string name="C++";
    int age{20};
    double score{99.5};
};
void printStudent(student s) {
    cout << s.name << endl;
    cout << s.age << endl;
    cout << s.score << endl;
}
int main() {
    student s1 = {"王五", 21, 99};
    printStudent(s1);
    return 0;
}
```

结构体做函数参数 案例

```
void printStudent(student *s) {  
    cout << s->name << endl;  
    cout << s->age << endl;  
    cout << s->score << endl;  
}
```

```
student s1 = { "王五", 21, 99 };  
printStudent2(&s1);
```



结构体做函数参数 案例

```
void printStudent(student s[],int len) {  
    for (int i = 0; i < len; i++) {  
        cout << s[i].name << " " << s[i].age << " " << s[i].score << endl;  
    }  
}
```

```
student s1[2] = {"王五",21,99,"张三",19,90};
```

```
printStudent(s1,2);
```



结构体数组

共用体

共用体可以存放任意 数据类型 的组合，也可以用来存放任意数据类型的集合；共用体也叫联合体。

结构体的各个成员会占用不同的内存，互相之间没有影响，而共用体的所有成员占用同一段内存，修改一个成员会影响其余所有成员。

共用体

语法

```
union unionName{  
    Type1 filed1;  
    Type2 filed2;  
    Type3 filed3;  
};
```

该共用体的三个字段是相互影响的，修改其中任何一个字段的值，都会影响其他字段的值

共用体成员访问

语法

```
unionName.fileId;
```

共用体案例

```
union person {  
    int age;  
    float weight;  
};
```

```
person p1;
```

```
p1.age = 20;
```

```
//p1.weight = 50.5; //共用体的成员不可以全部一起使用，只能单独使用其中某一个字段。
```

```
cout << p1.age << endl;
```

```
cout << p1.weight << endl; //共用体成员间影响内存
```

共用体与结构体区别

共用体和结构体都可以存放任意数据类型的组合，但两者在内存的使用上却有很大的差别。

结构体的各个成员会占用不同的内存，互相之间没有影响，而共用体的所有成员占用同一段内存，修改一个成员会影响其余所有成员。

结构体占用的内存大于等于所有成员占用的内存的总和（成员之间可能会存在缝隙），共用体占用的内存等于最长的成员占用的内存。

共用体使用了内存覆盖技术，同一时刻只能保存一个成员的值，如果对新的成员赋值，就会把原来成员的值覆盖掉

共用体与结构体区别

```
int sizeStruct = sizeof(struct StructName);  
int sizeUnion = sizeof(union UnionName);  
cout << "sizeStruct = " << sizeStruct << " sizeUnion = " << sizeUnion << endl;
```

枚举类型

枚举类型(enumeration)是 C++ 中的一种派生数据类型，它是由用户定义的若干枚举常量的集合。

语法

```
enum <类型名> {<枚举常量表>;
```

- 1.关键字enum：指明其后的标识符是一个枚举类型的名字。
- 2.枚举常量表：由枚举常量构成。

"枚举常量"或称"枚举成员"，是以标识符形式表示的整型量，表示枚举类型的取值。枚举常量表列出枚举类型的所有取值，各枚举常量之间以"，"间隔，且必须各不相同。取值类型与条件表达式相同。

枚举类型的定义

```
enum color {RED, BLUE, WHITE, BLACK}; // 定义枚举类型color
```

```
enum week {Sun, Mon, Tue, Wed, Thu, Fri, Sat}; // 定义枚举类型week
```

枚举类型的定义

枚举常量代表该枚举类型的变量可能取的值，编译系统为每个枚举常量指定一个整数值。

默认状态下，这个整数就是所列举元素的序号，序号从0开始。

可以在定义枚举类型时为部分或全部枚举常量指定整数值，在指定值之前的枚举常量仍按默认方式取值，而指定值之后的枚举常量按依次加1的原则取值。

各枚举常量的值可以重复。

```
enum fruit {apple, orange, banana=1, peach, grape}
```

//枚举常量apple=0,orange=1, banana=1,peach=2,grape=3。

```
enum week {Sun=7, Mon=1, Tue, Wed, Thu, Fri, Sat};
```

//枚举常量Sun,Mon,Tue,Wed,Thu,Fri,Sat的值分别为7、 1、 2、 3、 4、 5、 6

枚举类型的定义

枚举常量只能以标识符形式表示，而不能是整型、字符型等文字常量。

`enum letter {'a','b','c'};` //枚举常量不能是字符常量

`enum year{2022,2023,2024,2025};` //枚举常量不能是整型常量

`enum letter {a,b,c};`

`enum year{y2022,y2023,y2024,y2025};`

使用枚举变量

定义枚举类型之后，就可以定义该枚举类型的变量

语法

```
enumName enumVarName;
```

枚举名 枚举变量名；

使用枚举变量

```
enum week {Sun, Mon, Tue, Wed, Thu, Fri, Sat};
```

```
week w1 , w2 ;
```

类型与变量同时定义，类型名可省略

```
enum {Sun, Mon, Tue, Wed, Thu, Fri, Sat} w1 , w2 ;
```

使用枚举变量

枚举变量的值只能取枚举常量表中所列的值，就是整型数的一个子集。

枚举变量占用内存的大小与整型数相同。

枚举变量只能参与赋值和关系运算以及输出操作，参与运算时用其本身的整数值

使用枚举变量

```
enum week {Sun, Mon, Tue, Wed, Thu, Fri, Sat};
```

```
week weekday1, weekday2;
```

```
weekday1 = Sun; //正确
```

```
//weekday1 = San ; //错误 只能取枚举常量表中所列的值，就是整型数的一个子集。
```

使用枚举变量

```
enum week {Sun, Mon, Tue, Wed, Thu, Fri, Sat};
```

```
week weekday1, weekday2;
```

```
weekday1 = Sun;
```

```
weekday2 = Mon;
```

```
if(weekday1 < weekday2){ //只能参与赋值和关系运算
```

```
    cout << "day1 < day2";
```

```
}
```

```
cout << weekday2 << endl; //输出操作
```

```
cout << weekday1 + weekday2 << endl;
```

```
int num = weekday1;
```

枚举类型注意事项

枚举变量可以直接输出，但不能直接输入。

不能直接将常量赋给枚举变量。

不同类型的枚举变量之间不能相互赋值。

枚举变量的输入输出一般都采用switch语句将其转换为字符或字符串。

用typedef定义类型

typedef是C/C++中一个很关键的关键字，它的作用是用来给类型**起别名**。因为在遇上不同数据类型，尤其是复杂类型。

typedef关键字常用来给以下几种类型起别名：

1. 基本数据类型
2. 结构体
3. 数组
4. 枚举类型
5. 指针

用typedef定义类型

1. 基本数据类型

使用typedef给基本数据类型起别名很简单：

//typedef关键字 数据类型 别名

```
typedef int Myint;
```

```
typedef double Mydouble;
```

起了别名以后，别名就相当于原类型名的同义词，能使用原类型的地方就可以使用别名。

int num和Myint num等价。

用typedef定义类型

2. 结构体

和基本数据类型类似，将别名放在结构体之后

```
struct Person{  
    string name;  
    int age;  
};
```

//typedef关键字 结构体 别名

```
typedef struct Person P;
```

用typedef定义类型

3. 数组

给数组起别名时，typedef的用法和单个元素有些区别，**别名被挤在数组元素类型和数组元素个数之间。**

//typedef关键字 数组元素类型 别名[数组元素个数]

```
typedef int C[10];
```

C t; // t的类型是int[10]，也就是包含10个int类型的数组

用typedef定义类型

4. 枚举类型

和结构体类型类似

```
enum Color{Red,Green,Blue} primaryColor;
```

//typedef关键字 枚举类型 别名

```
typedef Color primaryColor;
```

```
primaryColor G = Red
```

用typedef定义类型

4. 枚举类型

和结构体类型类似

```
enum Color{Red,Green,Blue} primaryColor;
```

//typedef关键字 枚举类型 别名

```
typedef Color primaryColor;
```

```
primaryColor G = Red
```

用typedef定义类型

5. 指针

//typedef关键字 指针 别名

```
typedef char* String;
```

```
String s1= "name";
```

对指针起别名需要注意的一点就是，如果要对别名加const关键字，必须要知道，我们声明的是一个常量指针，而不能将其按照字面上，理解为指向常量的指针。

用typedef定义类型

5. 指针

//typedef关键字 指针 别名

```
typedef char* String;
```

```
String s1= "name";
```

对指针起别名需要注意的一点就是，如果要对别名加const关键字，必须要知道，我们声明的是一个常量指针，而不能将其按照字面上，理解为指向常量的指针。

案例1题目

利用结构体，输入n ($2 < n < 100$)个学生姓名，年龄，和三科 (语文，数学，英语)成绩，找出总成绩最高的学生，输出该学生的信息。

案例1 代码

```
#include <iostream>
using namespace std;
struct student {
    string name;
    int age;
    double chinese;
    double math;
    double english;
    double total;
} s[100];
```

```
int main() {
    int n;
    double maxv = 0;
    student s2;
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> s[i].name >> s[i].age >> s[i].chinese >> s[i].math >> s[i].english;
        s[i].total = s[i].chinese + s[i].math + s[i].english;
    }
    for (int i = 0; i < n; i++)
    {
        if (s[i].total > maxv) {
            maxv = s[i].total;
            s2 = s[i];
        }
    }
    cout << s2.name << " " << s2.age << " " << s2.chinese << " "
        << s2.math << " " << s2.english << " "
        << s2.total << endl;
    return 0;
}
```

案例2题目

输入数字0-6,0表示星期日，通过switch和enum实现，输出星期对应汉字表述。

输入用例：0

输出用例：星期日

案例2 代码

```
enum week { Sun, Mon, Tue, Wed, Thu, Fri, Sat }; // 定义枚举类型week
int n;
cin >> n;
switch (n)
{
case Sun:
    cout << "星期日" << endl;
    break;
case Mon:
    cout << "星期一" << endl;
    break;
case Tue:
    cout << "星期二" << endl;
    break;
case Wed:
    cout << "星期三" << endl;
    break;
case Thu:
    cout << "星期四" << endl;
    break;
case Fri:
    cout << "星期五" << endl;
    break;
case Sat:
    cout << "星期六" << endl;
    break;
}
```

知识点总结

- | | |
|-----------------|----|
| 1. 结构体概述 | 了解 |
| 2. 结构体的定义和初始化 | 重点 |
| 3. 结构体指针 | 重点 |
| 4. 结构体数组 | 重点 |
| 5. 结构体做函数参数 | 重点 |
| 6. 共用体 | 熟悉 |
| 7. 枚举类型 | 熟悉 |
| 8. 用typedef定义类型 | 熟悉 |

作业

学校正在做毕设项目，每个老师要求带5个学生，总共有三个老师，需求如下：

设计学生和老师的结构体，其中在老师的结构体中，有老师姓名和一个存放5名学生的数组作为成员，学生的成员有姓名考试分数，创建数组存放三名老师，通过函数给每个老师及所带的学生赋值，最终打印出老师数据及老师所带的学生数据。

下节课预告

预处理命令